AI - Assisted Coding-Assignment -13.2

HTNO:2303A52228
Batch:37

Lab 13 - Code Refactoring – Improving Legacy Code with AI Suggestions.

Task Description-1
Refactoring – Eliminating Repetitive Logic.
Expected Output:
A refactored Python script that uses a single reusable function to
compute area and perimeter without repeating code.
Prompt:
#write a python code to find area and perimeter taking the input as length and width
Function to calculate area and perimeter and rreturn the output
#encapsulate repeated logic into resuable functions
#add proper docstrings to the functions.
CODE AND OUTPUT :

13.py > main

```python
def calculate_properties(length, width):
    """
    Calculate both area and perimeter
    Args:
        length (float): length
        width (float): width
    Returns:
        tuple: A tuple containing (area, perimeter).
    """
    area = length * width
    perimeter = 2 * (length + width)
    return area, perimeter
def main():
    """Main function to get user input and display area and perimeter."""
    length = float(input("Enter the length: "))
    width = float(input("Enter the width : "))
    area, perimeter = calculate_properties(length, width)
    print(f"Area : {area}")
    print(f"Perimeter : {perimeter}")
if __name__ == "__main__":
    main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
& C:\Users\ajayr\AppData\Local\Programs\Python\Python313\python.exe c:/AIAC/13.py
Enter the length: 8
Enter the width : 4
Area : 32.0
Perimeter : 24.0
```

Explanation:
The original program had repeated calculations for finding the area and perimeter of a rectangle using different values of length and width. To remove this repetition, the logic was refactored by creating a reusable function called calculate_rectangle(length, width). This function performs the calculations once and can be called multiple times with different inputs. A docstring was also added to explain the purpose, parameters, and behavior of the function, making the code more readable and maintainable.

Task Description-2
Refactoring – Modularizing Inline Calculations.
Expected Output:
A modular Python program that uses a reusable function to compute final amounts for different inputs.
Prompt:
#refactor a Python script by extracting repeated inline calculations

into reusable and well-documented functions.
#expected output should be modular Python program that uses a reusable function to
#compute final amounts for different inputs userinput

CODE :

13.py > ...

```python
#refactor a Python script by extracting repeated inline calculations
# into reusable and well-documented functions.
#expected output should be modular Python program that uses a reusable f
#final amounts for different inputs userinput
def calculate_final_amount(amount, discount_percent):
    """
    Calculate the final amount after applying a discount.
    Parameters:
    amount (float): The original amount
    discount_percent (float): The discount percentage (0-100)
    Returns:
    float: The final amount after discount is applied
    """
    discount = amount * (discount_percent / 100)
    final_amount = amount - discount
    return final_amount
# Example usage
if __name__ == "__main__":
    amount = float(input("Enter the amount: "))
    discount_percent = float(input("Enter the discount percentage: "))
    final_amount = calculate_final_amount(amount, discount_percent)
    print(f"Final Amount: {final_amount:.2f}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
& C:\Users\ajayr\AppData\Local\Programs\Python\Python313\python.exe c:/AIAC/13.py
Enter the amount: 1000
Enter the discount percentage: 0.10
Final Amount: 999.00
```

Explanation:

The function just computes a discount and returns the reduced amount.
The main block prompts for figures, converts them to floats, calls that helper and prints the result formatted to two decimal places.
That's it – core logic in a reusable function, I/O under the script guard.

Task Description-3

Refactoring – Improving Loop and Conditional Performance.

Expected Output:

An optimized Python program that performs faster lookups using efficient logic, along with execution performance comparison.

Prompt:

#enhance the performance of Python code that relies on nested loops

#use more efficient data structures or logic
#Replace inefficient constructs while preserving program behavior.
#Compare execution efficiency before and after refactoring.
#expected outputshould be An optimized Python program that performs faster
#efficient logic, along with execution performance comparison.

CODE :

13.py > ...

```python
import time
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]
start = time.time()
for name in check_list:
    exists = False
    for student in students:
        if name == student:
            exists = True
    if exists:
        print(name, "found")
    else:
        print(name, "not found")
inefficient_time = time.time() - start
print("\n--- Optimized approach ---\n")
start = time.time()
students_set = set(students)
for name in check_list:
    if name in students_set:
        print(name, "found")
    else:
        print(name, "not found")
optimized_time = time.time() - start
print(f"\nInefficient approach: {inefficient_time:.6f}s")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
PS C:\AIAC> & C:\Users\ajayr\AppData\Local\Programs\Python\Python313\python.exe c:/AIAC/13.py
Inefficient approach: 0.001430s
Optimized approach: 0.002546s
Speedup: 0.56x faster
```

Explanation:

This program compares two ways of checking whether names exist in a student list. The first approach uses nested loops, which checks every name in the list one by one, making it slower for large data. The optimized approach converts the list into a set, which allows faster membership checking using in. The execution time for both methods is measured, and the program shows that the set-based approach is much faster and more efficient.

Task Description-4

Refactoring – Replacing Hardcoded Values with Constants.

Expected Output:

A revised Python program using constants such as RATE and TIME for

clearer and maintainable calculations.

Prompt:

Refactor the given Python code by identifying hardcode numeric values and replacing them with descriptive constants. Define these constants at the beginning of the program and update the calculations to use them. Ensure that the output of the code.

CODE :

13.py > ...

```python
# Define constants
RATE = 2
TIME = 5
print("Simple Interest:", (1000 * RATE * TIME) / 100)
print("Simple Interest:", (2000 * RATE * TIME) / 100)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
PS C:\AIAC>
& C:\Users\ajayr\AppData\Local\Programs\Python\Python313\python.exe c:/AIAC/13.py
Simple Interest: 100.0
Simple Interest: 200.0
```

Explanation:

In the original program, the numbers 2 and 5 were directly used in the calculation. These are called hardcoded values, which can make the code harder to understand or update later. To improve this, we created constants named RATE and TIME at the beginning of the program. Now the formula uses these constants, making the code clearer and easier to maintain if the values ever need to change.

Task Description-5

Re factoring – Enhancing Variable Naming and Code Clarity.

Expected Output:

A Python program with clear variable names and explanatory comments that improve readability without altering functionality.

Prompt:

Refactor the given Python code by improving variable names and adding meaningful comments. Replace unclear variable names with descriptive ones and include inline comments to explain the logic. Ensure that the program output remains the same.

CODE :

13.py > ...

```python
# Assign values to two numbers
first_number = 50
second_number = 30
# Calculate the sum of the two numbers
total_sum = first_number + second_number
# Display the result
print(total_sum)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
PS C:\AIAC>
& C:\Users\ajayr\AppData\Local\Programs\Python\Python313\python.exe c:/AIAC/13.py
80
```

Explanation:
In the original code, the variables x, y, and z were not very descriptive, so it was difficult to understand what they represented. In the refactored version, these variables were renamed which clearly describe their purpose. Comments were also added to explain each step of the program. This improves the readability and clarity of the code while keeping the output exactly the same.